

Omni-Channel Developer Guide

Version 67.0, Summer '26



Summer '26

CONTENTS

- Chapter 1: Omni-Channel Developer Guide 1**
 - Omni-Channel Objects 2
 - Omni-Channel Metadata API Types 2
 - Omni-Channel Components for the Salesforce Console 3
 - Lightning Console JavaScript API 3
 - Salesforce Console Integration Toolkit 4
 - External Routing for Omni-Channel 8
 - Technical Architecture and Process 8
 - Integrate External Routing 9
 - Expected Behavior 16
 - Troubleshooting 17
- Index 19**

CHAPTER 1 Omni-Channel Developer Guide

In this chapter ...

- [Omni-Channel Objects](#)
- [Omni-Channel Metadata API Types](#)
- [Omni-Channel Components for the Salesforce Console](#)
- [External Routing for Omni-Channel](#)

Customize your Omni-Channel records and console integration with Omni-Channel API objects and console methods.



Important: Standard Omni-Channel reaches its End of Life (EOL) with the Summer '26 release. For continued support, access to the latest features, and an enhanced experience, update to [Enhanced Omni-Channel](#).

Omni-Channel routes work items to queues, agents, skills, and even Einstein Bots (on supported channels) based on defined routing logic. Depending on your business needs, you can use different ways to route work, including Omni-Channel flows.

Use Omni-Channel to manage the priority of work items, which makes it a cinch to route important work items to agents quickly. Manage your agents' capacity to take on work items so that they're given only the number of assignments that they can handle. You can also define which agents can work on different types of assignments. For example, you can create one group of agents to respond to leads and sales inquiries, and another group that helps customers with support questions.

Routing logic is applied when work is assigned to an owner. If field values on the work item are changed after the item is routed, the routing logic isn't reapplied.

Omni-Channel Objects

Use an API to create, retrieve, update or delete records, such as accounts, leads, and custom objects. The Salesforce data model includes several objects that let you control and customize your Omni-Channel records, including Omni-Channel users, routing configurations, and statuses.

For more information on Salesforce APIs, see [Which API Do I Use?](#) in *Salesforce Help*.

- [AgentWork](#)
- [AgentWorkSkill](#)
- [OmniSupervisorConfig](#)
- [OmniSupervisorConfigAction](#)
- [OmniSupervisorConfigGroup](#)
- [OmniSupervisorConfigProfile](#)
- [OmniSupervisorConfigQueue](#)
- [OmniSupervisorConfigSkill](#)
- [OmniSupervisorConfigTab](#)
- [OmniSupervisorConfigUser](#)
- [PendingServiceRouting](#)
- [PresenceConfigDeclineReason](#)
- [PresenceDeclineReason](#)
- [PresenceUserConfig](#)
- [PresenceUserConfigUser](#)
- [QueueRoutingConfig](#)
- [QueueSubject](#)
- [ServiceChannel](#)
- [ServiceChannelFieldPriority](#)
- [ServiceChannelStatus](#)
- [ServicePresenceStatus](#)
- [ServiceResource](#)
- [SkillRequirement](#)
- [UserServicePresence](#)

Omni-Channel Metadata API Types

The Metadata API lets you access Omni-Channel feature settings and metadata information.

The following types are available with the Metadata API.

- [Flow](#)
- [OmniChannelSettings](#)
- [OmniSupervisorConfig](#)
- [PresenceDeclineReason](#)
- [PresenceUserConfig](#)

- [Queue](#)
- [QueueRoutingConfig](#)
- [ServiceChannel](#)
- [ServicePresenceStatus](#)
- [WorkSkillRouting](#)
- [WorkSkillRoutingAttribute](#)

If you need more information on the Salesforce Metadata API, see the [Metadata API Developer Guide](#).

Omni-Channel Components for the Salesforce Console

Omni-Channel lets your call center route any type of incoming work item to the most qualified, available agents.

[Omni-Channel Components for the Lightning Console JavaScript API](#)

Omni-Channel lets your call center route any type of incoming work item to the most qualified, available agents. The Lightning Console JavaScript API for Lightning Experience includes several methods and events that let you control how Omni-Channel works within the Lightning Service Console for your organization.

[Omni-Channel Components for the Salesforce Console Integration Toolkit](#)

The Salesforce Console Integration Toolkit includes several objects that let you control how Omni-Channel works within the Salesforce console for your organization. If your org is using Salesforce Classic, use Salesforce Console Integration Toolkit methods.

Omni-Channel Components for the Lightning Console JavaScript API

Omni-Channel lets your call center route any type of incoming work item to the most qualified, available agents. The Lightning Console JavaScript API for Lightning Experience includes several methods and events that let you control how Omni-Channel works within the Lightning Service Console for your organization.

If you need more information on the Lightning Console JavaScript API, see [Lightning Console JavaScript API](#).

[Methods for Omni-Channel in Lightning Experience](#)

Use these Lightning Console JavaScript API methods for Omni-Channel. Omni-Channel lets your call center route any type of incoming work item to the most qualified, available agents.

[Events for Omni-Channel in Lightning Experience](#)

Use these Lightning Console JavaScript API events for Omni-Channel. JavaScript can be executed when certain types of events occur in a console, such as when a user closes a tab. There are a few events that are specific to Omni-Channel. These events apply to Lightning Experience only.

Methods for Omni-Channel in Lightning Experience

Use these Lightning Console JavaScript API methods for Omni-Channel. Omni-Channel lets your call center route any type of incoming work item to the most qualified, available agents.

- [acceptAgentWork for Lightning Experience](#)
- [closeAgentWork for Lightning Experience](#)
- [declineAgentWork for Lightning Experience](#)
- [getAgentWorkload for Lightning Experience](#)

- [getAgentWorks](#) for Lightning Experience
- [getServicePresenceStatusChannels](#) for Lightning Experience
- [getServicePresenceStatusId](#) for Lightning Experience
- [login](#) for Lightning Experience
- [logout](#) for Lightning Experience
- [lowerAgentWorkFlag](#) for Lightning Experience
- [raiseAgentWorkFlag](#) for Lightning Experience
- [setServicePresenceStatus](#) for Lightning Experience

Events for Omni-Channel in Lightning Experience

Use these Lightning Console JavaScript API events for Omni-Channel. JavaScript can be executed when certain types of events occur in a console, such as when a user closes a tab. There are a few events that are specific to Omni-Channel. These events apply to Lightning Experience only.

- [lightning:omniChannelConnectionError](#)
- [lightning:omniChannelLoginSuccess](#)
- [lightning:omniChannelStatusChanged](#)
- [lightning:omniChannelLogout](#)
- [lightning:omniChannelWorkAssigned](#)
- [lightning:omniChannelWorkAccepted](#)
- [lightning:omniChannelWorkDeclined](#)
- [lightning:omniChannelWorkClosed](#)
- [lightning:omniChannelWorkFlagUpdated](#)
- [lightning:omniChannelWorkloadChanged](#)

Omni-Channel Components for the Salesforce Console Integration Toolkit

The Salesforce Console Integration Toolkit includes several objects that let you control how Omni-Channel works within the Salesforce console for your organization. If your org is using Salesforce Classic, use Salesforce Console Integration Toolkit methods.

If you need more information on the Salesforce Console Integration Toolkit, see [Salesforce Console Integration Toolkit for Salesforce Classic](#).

[Methods for Omni-Channel for Salesforce Console Integration Toolkit](#)

Use these Salesforce Console Integration Toolkit API methods for Omni-Channel in Salesforce Classic.

[Methods for Console Events](#)

JavaScript can be executed when certain types of events occur in a console, such as when a user closes a tab. In addition to the standard methods for console events, there are a few events that are specific to Omni-Channel. These events apply to Salesforce Classic only.

Methods for Omni-Channel for Salesforce Console Integration Toolkit

Use these Salesforce Console Integration Toolkit API methods for Omni-Channel in Salesforce Classic.

- [acceptAgentWork](#)

- [closeAgentWork](#)
- [declineAgentWork](#)
- [getAgentWorks](#)
- [getAgentWorkload](#)
- [getServicePresenceStatusChannels](#)
- [getServicePresenceStatusId](#)
- [login](#)
- [logout](#)
- [setServicePresenceStatus](#)
- [Methods for Omni-Channel Console Events](#)

Methods for Console Events

JavaScript can be executed when certain types of events occur in a console, such as when a user closes a tab. In addition to the standard methods for console events, there are a few events that are specific to Omni-Channel. These events apply to Salesforce Classic only.

Standard Console Events

Event	Description	Payload
<code>sforce.console.ConsoleEvent.OPEN_TAB</code>	Fired when a primary tab or subtab is opened. Available in API version 30.0 or later.	• <code>id</code>—the ID of the opened tab • <code>objectId</code>—the object ID of the opened tab, if available
<code>sforce.console.ConsoleEvent.CLOSE_TAB</code>	Fired when a primary tab or subtab with a specified ID in the <code>additionalParams</code> argument is closed. Or, fired when a primary tab or subtab with no specified ID is closed. Available in API version 30.0 or later.	• <code>id</code>—the ID of the closed tab • <code>objectId</code>—the object ID of the closed tab, if available
<code>sforce.console.ConsoleEvent.CONSOLE_LOGOUT</code>	Delays the execution of logging out of a console when a user clicks Logout. When Logout is clicked: 1. An overlay appears, which tells a user that logout is in progress. 2. Callbacks are executed that have been registered by using <code>sforce.console.ConsoleEvent.CONSOLE_LOGOUT</code> 3. Console logout logic is executed. If the callback contains synchronous blocking code, the console logout code isn't executed until the blocking code is executed. As a best practice, avoid synchronous blocking code or long code execution during logout. Available in API version 31.0 or later.	None

Omni-Channel Console Events

Event	Description	Payload
<code>sforce.console.ConsoleEvent.PRESENCE.LOGIN_SUCCESS</code>	Fired when an Omni-Channel user logs into Omni-Channel successfully. Available in API version 32.0 or later.	<code>statusId</code>—the ID of the agent's current presence status.
<code>sforce.console.ConsoleEvent.PRESENCE.STATUS_CHANGED</code>	Fired when a user changes his or her presence status. Available in API version 32.0 or later.	<code>statusId</code>—the ID of the agent's current presence status. <code>channels</code>—channel JSON string of channel objects. <code>statusName</code>—the name of the agent's current presence status. <code>statusApiName</code>—the API name of the agent's current presence status.
<code>sforce.console.ConsoleEvent.PRESENCE.LOGOUT</code>	Fired when a user logs out of Salesforce. Available in API version 32.0 or later.	None
<code>sforce.console.ConsoleEvent.PRESENCE.WORK_ASSIGNED</code>	Fired when a user is assigned a new work item. Available in API version 32.0 or later.	<code>workItemId</code>—the ID of the object that's routed through Omni-Channel. This object becomes a work assignment with a <code>workId</code> when it's assigned to an agent. <code>workId</code>—the ID of a work assignment that's routed to an agent.
<code>sforce.console.ConsoleEvent.PRESENCE.WORK_ACCEPTED</code>	Fired when a user accepts a work assignment, or when a work assignment is automatically accepted. Available in API version 32.0 or later.	<code>workItemId</code>—the ID of the object that's routed through Omni-Channel. This object becomes a work assignment with a <code>workId</code> when it's assigned to an agent. <code>workId</code>—the ID of a work assignment that's routed to an agent.
<code>sforce.console.ConsoleEvent.PRESENCE.WORK_DECLINED</code>	Fired when a user declines a work assignment. Available in API version 32.0 or later.	<code>workItemId</code>—the ID of the object that's routed through Omni-Channel. This object becomes a work assignment with a <code>workId</code> when it's assigned to an agent. <code>workId</code>—the ID of a work assignment that's routed to an agent.

Event	Description	Payload
<code>sforce.console.ConsoleEvent.PRESENCE.WORK_CLOSED</code>	Fired when the status of an AgentWork object is changed to Closed. Available in API version 32.0 or later.	<code>workItemId</code>—the ID of the object that's routed through Omni-Channel. This object becomes a work assignment with a <code>workId</code> when it's assigned to an agent. <code>workId</code>—the ID of a work assignment that's routed to an agent.
<code>sforce.console.ConsoleEvent.PRESENCE.WORKLOAD_CHANGED</code>	Fired when an agent's workload changes. This includes receiving new work items, declining work items, and closing items in the console. It's also fired when there's a change to an agent's capacity or Presence Configuration or when the agent goes offline in the Omni-Channel widget.	<code>ConfiguredCapacity</code>—the configured capacity for the agent. <code>PreviousWorkload</code>—the agent's workload before the change. <code>NewWorkload</code>—the agent's new workload after the change.

channel

The `channel` object contains the following properties:

Name	Type	Description
<code>channelId</code>	String	Retrieves the ID of a service channel that's associated with a presence status.
<code>developerName</code>	String	Retrieves the developer name of the the service channel that's associated with the <code>channelId</code> .

Methods for Console Events

Method	Description
addEventListener()	Adds a listener for a custom event type or a standard event type when the event is fired. This method adds a listener for custom event types in API version 25.0 or later; it adds a listener for standard event types in API version 30.0 or later.
fireEvent()	Fires a custom event. This method is only available in API version 25.0 or later.
removeEventListener()	Removes a listener for a custom event type or a standard event type. This method removes a listener for custom event types in API version 25.0 or later; it removes a listener for standard event types in API version 30.0 or later.

External Routing for Omni-Channel

Multiple routing options, one console. Integrate third-party routing with Omni-Channel to give your support team more routing options for their work.

Before setting up external routing, make sure that you have a working implementation of Omni-Channel. You'll use version 41.0 or later of Salesforce standard APIs and streaming APIs to connect Salesforce with an external routing implementation in your routing configuration. Then you can create queues that use either Omni-Channel routing or your external routing implementation.

 **Note:** With external routing, Skills-Based routing or direct-to-rep routing is not supported.

Are you ready to set up and use external routing? Let's get started.

[External Routing Technical Architecture and Process](#)

See an overview of how external routing works to connect Salesforce with your external routing implementation.

[Expected Behavior for External Routing for Omni-Channel](#)

Verify that the behavior you observe while testing and using your implementation of external routing matches the following expected behavior scenarios.

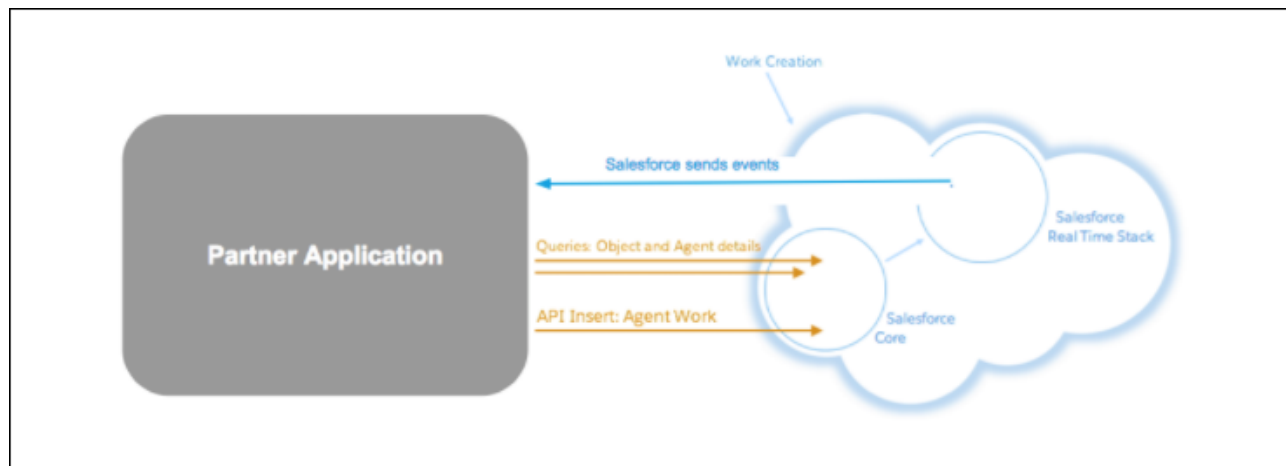
[Troubleshooting External Routing for Omni-Channel](#)

If you encounter issues with your implementation of External Routing for Omni-Channel, try the following troubleshooting steps.

External Routing Technical Architecture and Process

See an overview of how external routing works to connect Salesforce with your external routing implementation.

The following image provides a visual description of how information is shared between Salesforce and your partner application.



Information is shared using Salesforce APIs and the APIs for your partner application using the following process.

1. Salesforce uses Change Data Capture to send event notifications to reps .
2. Salesforce sends event notifications for new work items to the partner routing application.
3. Partner routing application matches new work items with logged-in reps.

4. Salesforce sends event notifications to the partner routing application after work is accepted, declined, or completed.
5. Salesforce sends event notifications to the partner routing application after reps log out.

[Salesforce API Resources for External Routing](#)

Use the following resources as you integrate your partner application with Omni-Channel.

Salesforce API Resources for External Routing

Use the following resources as you integrate your partner application with Omni-Channel.

- [AgentWork](#)
- [PendingServiceRouting](#)
- [UserServicePresence](#)
- [Streaming API Developer Guide](#)
- [Streaming API Message Durability](#)
- [Streaming API Limits](#)

Integrate External Routing for Omni-Channel

Use the following steps to integrate your external routing implementation with Omni-Channel.

1. [Create a Routing Configuration and Queue for External Routing.](#)
2. [Add Objects to Change Data Capture for External Routing.](#)
3. [Subscribe to Change Data Capture Event Notifications.](#)
4. [Create AgentWork.](#)

Create a Routing Configuration and Queue for External Routing

External routing requires a separate routing configuration and queue in Omni-Channel. These separate objects define routing behavior and assign work to reps .

1. In Setup, enter *Routing Configurations* in the Quick Find box, then select **Routing Configurations**.
2. Create a routing configuration and select **External Routing** for the routing model.
3. Enter *Queues* in the Quick Find box, then select **Queues**.
4. Create a queue and connect it to the routing configuration you created.

SEE ALSO:

[Create Routing Configurations for Your Queues](#)

[Create a Queue](#)

Add Objects to Change Data Capture for External Routing

To enable external routing with Omni-Channel, configure Change Data Capture (CDC) to publish event notifications for core Service Cloud objects.

To add objects to CDC:

- From Setup, in the Quick Find box, enter *Change Data Capture*, and then select **Change Data Capture**.
- In the **Available Entities** list, select these objects to set up external routing with Omni-Channel.
 - `PendingServiceRouting`: Routing details for a work item awaiting routing or assignment.
 - `AgentWork`: Work assignment routed to a rep.
 - `UserServicePresence`: A presence user's real-time presence status.
- Move the selected objects to the **Selected Entities** list.

You can select up to five entities, including standard and custom objects. To enable more entities, contact your Salesforce Account Representative to purchase an add-on license.

The partner routing application must monitor the following event types for the `PendingServiceRouting` record:

- Creating the Pending Service Record (PSR) prompts the partner routing application to search for a rep.
- Update event notifications signal a change in record status. The partner routing application can check for updates that indicate the `IsPushed` value has changed.
 - If the `IsPushed` value changes to `true`, an `AgentWork` record was created for the PSR.
 - If the `IsPushed` value changes to `false`, the work was declined and must be assigned to another rep.
- A Delete event notification indicates that the work was accepted by a rep or that the reason for creating the work was removed. For example, the customer hung up the phone or ended a conversation. Upon receiving the Delete event notification, stop tracking the PSR and remove it from the system.

The partner routing application must monitor the following event types for the `UserServicePresence` record:

- Tracking the creation of this record allows the partner routing application to track the status of each rep.
- An Update event notification showing the `IsCurrentState` updated to `false` indicates the rep has changed their status or has gone offline.
 - If the rep has changed their status, a Create record event notification is generated immediately after the Update event notification
 - If the rep has logged out, no Create record event notification is generated following the Update event notification

The partner routing application must monitor the following event types for the `AgentWork` record:

- The external routing system creates the `AgentWork` record when a work item is routed to a rep. The notification can be treated as a success message.
- The external routing system must track Update event notifications, focusing on changes to the `Status` fields to monitor whether a rep accepts or declines a work item.
 - Work items are created in the `Assigned` status.
 - The status changes to `Opened` after the work is accepted.
 - An update indicating the `Unavailable` status means a rep went offline while they had `Assigned` work items. A new rep needs to be assigned to the work items.
 - An update indicating the `Declined` or `DeclinedOnPushTimeout` status means the rep has declined the work. A new rep needs to be assigned to the work items.
 - All other statuses indicate that the work was completed or is no longer needed. The rep's capacity can be freed up and new work can be assigned to the rep.

Subscribe to Change Data Capture Event Notifications

Select a subscription method to track Change Data Capture (CDC) event notifications and set up a listener for real-time routing data. Event notifications provide real-time updates to the partner routing application for data synchronization and routing decisions.

You have two options to subscribe to CDC event notifications and listen to case routing data:

- Subscribe using Pub/Sub API
- Subscribe using Apex Triggers

Subscribe Using Pub/Sub API

With Pub/Sub API you can subscribe to event notifications in an external client to integrate your systems. You can create custom channels to receive event notifications specifically for create, update, and delete events necessary for external routing. To receive all updates about external routing:

- Create a custom channel. The new channel's name must follow this structure:

```
ExternalRoutingChannel__chn.platformEventChannel
```

- Define the channel members. This channel receives CDC event notifications for objects critical to external routing, such as `PendingServiceRouting`.
- Configure the channel members. This example code provides a sample channel structure definition.

```
<?xml version="1.0" encoding="UTF-8"?>
<PlatformEventChannel xmlns="http://soap.sforce.com/2006/04/metadata">
  <channelType>data</channelType>
  <label>External Routing Channel</label>
</PlatformEventChannel>
```

- Listen for event notifications.

To receive updates only when the records are ready for external routing:

- Create a custom channel. Define a channel member record that subscribes to the `PendingServiceRoutingChangeEvent`. Use this filename structure for the channel member metadata:

```
ExtPSRChangeEvent.platformEventChannelMember
```

- Define the channel members. The channel members must have (`ExternalRoutingChannel__chn`) as the destination channel, and `PendingServiceRoutingChangeEvent` as the monitored object.
- Configure the channel members. This example code provides a sample channel filter logic:

```
<?xml version="1.0" encoding="UTF-8"?> <PlatformEventChannelMember
xmlns="http://soap.sforce.com/2006/04/metadata">
  <eventChannel>ExternalRoutingChannel__chn</eventChannel>
  <selectedEntity>PendingServiceRoutingChangeEvent</selectedEntity>
  <filterExpression>RoutingModel = 'ExternalRouting' AND IsReadyForRouting =
true</filterExpression> </PlatformEventChannelMember>
```

This filter expression ensures that events are only delivered when the routing model is set to `ExternalRouting` and the `IsReadyForRouting` field is true.

- Listen for event notifications.

To ensure the external routing system receives notifications when a rep logs in, logs out, or changes status:

- Create a custom channel. Define a channel member record that subscribes to the `UserServicePresenceChangeEvent`. Use this filename structure for the channel member metadata:

```
ExtUSPUpdateEvent.platformEventChannelMember
```

- Define the channel members. The channel members must have (ExternalRoutingChannel__chn) as the destination channel , and UserServicePresenceChangeEvent as the monitored object.
- Configure the channel members. The channel member record should include this configuration, which is critical for allowing the external routing system to monitor rep availability:

```
<?xml version="1.0" encoding="UTF-8"?>
<PlatformEventChannelMember xmlns="http://soap.sforce.com/2006/04/metadata">
  <eventChannel>ExternalRoutingChannel__chn</eventChannel>
  <selectedEntity>UserServicePresenceChangeEvent</selectedEntity>
  <filterExpression>ChangeEventHeader.changeType = 'CREATE' OR (IsCurrentState =
false)</filterExpression>
</PlatformEventChannelMember>
```

- Listen for event notifications.

To filter for updates to externally routed AgentWork records:

- Create a custom channel. Define a channel member record that subscribes to the AgentWorkChangeEvent. Use this filename for the channel member metadata:

```
ExtAgentWorkChangeEvent.platformEventChannelMember
```

- Define the channel members. The channel members must have (ExternalRoutingChannel__chn) as the destination channel , and AgentWorkChangeEvent as the monitored object.
- Configure the channel members. The channel member record should include this configuration, which is necessary to monitor the real-time status and lifecycle of work items being managed by the partner routing application:

```
<?xml version="1.0" encoding="UTF-8"?>
<PlatformEventChannelMember xmlns="http://soap.sforce.com/2006/04/metadata">
  <eventChannel>ExternalRoutingChannel__chn</eventChannel>
  <selectedEntity>AgentWorkChangeEvent</selectedEntity>
  <filterExpression>ChangeEventHeader.changeType = 'UPDATE' AND RoutingModel =
'ExternalRouting'</filterExpression>
</PlatformEventChannelMember>
```

To receive the Pub/Sub API event notifications, the partner routing application must use the setting:
/data/ExternalRoutingChannel__chn.

Subscribe Using Apex Triggers

You can use Apex Triggers to capture event notifications. With Apex Triggers, it isn't possible to filter for specific events.

To implement a listener for real-time external routing, define an Apex trigger that runs whenever a change event is generated for a PSR record. This example shows an Apex trigger for the PendingServiceRoutingChangeEvent. This trigger identifies and prepares new work for processing by the partner routing application:

```
trigger PSR_CDC_Trigger on PendingServiceRoutingChangeEvent (after insert) {

  // Iterate through each event message.
  for (PendingServiceRoutingChangeEvent event : Trigger.New) {
    // Get some event header fields
    EventBus.ChangeEventHeader header = event.ChangeEventHeader;
    // All the PSR records that were updated in this transaction. Could be multiple
    at once - e.g. when updating due to User going offline with multiple work items assigned.
```

```

        List<String> recordIds = event.ChangeEventHeader.getRecordIds();
        System.debug('Received change event for ' + header.entityName + ' for ' +
header.recordIds + ' of type ' + header.changeType + ' operation.');
```

```

        if (header.changetype == 'CREATE')
        {
            // New PSR record - check model is of type external & isReadyForRouting = true
            before proceeding
            System.debug('event.IsReadyForRouting: ' + event.IsReadyForRouting);
            System.debug('event.RoutingModel: ' + event.RoutingModel);
            if (event.IsReadyForRouting && event.RoutingModel == 'ExternalRouting')
            {
                // Process new work - remember to iterate all record Ids
                // Likely want to send this to your routing engine to determine who to
route it to.
            }
        }

        // For update operations, we can get a list of changed fields
        else if (header.changetype == 'UPDATE') {
            // Updated record details. Updates we care about are:
            //   if 'IsReadyForRouting' updated to true - means it is now ready for routing, and
            needs an owner. Edge case - this is only where custom routing is used
            //   'LastDeclinedAgentSession' is set/updated - indicates work was declined
            //   if 'IsPushed' becomes true - means it was assigned. External system should
            know about this already as they create the AgentWork so can likely ignore.
            //   if 'IsPushed' becomes false - means it was released by assigned Agent. Either due
            to a decline (manual or push timeout), or they went offline (unavailable).
            System.debug('List of all changed fields:');
            // Process the update based on what changed - remember to iterate all record
            Ids
            for (String field : header.changedFields) {
                if (null == event.get(field)) {
                    System.debug('Deleted field value (set to null): ' + field);
                } else {
                    System.debug('Changed field value: ' + field + '. New Value: '
+ event.get(field));
                }
            }
        }

        else if (header.changetype == 'DELETE') {
            // Indicates that either the work was accepted, or is no longer needed (i.e. canceled).
            Either way, no longer need to track this work
            // Remember to iterate all record Ids
        }
    }
}

```

To allow the partner routing application to monitor agent status changes in real time, define an Apex trigger that executes automatically after an event is inserted into the event stream for the `UserServicePresence` object. This example shows an Apex trigger for the `UserServicePresenceChangeEvent`. This trigger monitors the presence status of reps and conveys them to the partner routing application:

```
trigger USP_CDC_Trigger on UserServicePresenceChangeEvent (after insert) {
```

```

// Iterate through each event message.
for (UserServicePresenceChangeEvent event : Trigger.New) {
    // Get some event header fields
    EventBus.ChangeEventHeader header = event.ChangeEventHeader;
    // All the USP records that were updated in this transaction. Would only ever
    expect 1 per event
    List<String> recordIds = event.ChangeEventHeader.getRecordIds();
    System.debug('Received change event for ' + header.entityName + ' for ' +
header.recordIds + ' of type ' + header.changeType + ' operation.');
```

if (header.changetype == 'CREATE')

```

    {
        // New USP record - indicates either a first login to Omni-Channel or a Presence
        Status change to a new presence status
        // Will want to call out to the 3rd party service to let them know about the
        state of this user (e.g. Busy/Away v Online etc.)

    }
    // For update operations, we can get a list of changed fields
    else if (header.changetype == 'UPDATE') {
        // Updated record. Updates we care about are:
        //   IsCurrentState - Will get updated to false when the user changes status
        or logs out.
        // Will also get events about 'AverageCapacity' changes as work is accepted &
        completed, but isn't relevant for External Routing so can be ignored
        System.debug('List of all changed fields:');
        for (String field : header.changedFields) {
            if (null == event.get(field)) {
                System.debug('Deleted field value (set to null): ' + field);
            } else {
                System.debug('Changed field value: ' + field + '. New Value: '
+ event.get(field));
            }
        }
    }
    else if (header.changetype == 'DELETE') {
        // Should never get here for UserServicePresence
    }
}
}

```

To help synchronize the state of assigned work items between Salesforce and the partner routing application, define an Apex trigger to capture and process CDC events for the `AgentWork` object. This example shows an Apex trigger for the `AgentWorkPresenceChangeEvent`. This trigger identifies when a new `AgentWork` record has been created:

```

trigger AgentWork_CDC_Trigger on AgentWorkChangeEvent (after insert) {
    // Iterate through each event message.
    for (AgentWorkChangeEvent event : Trigger.New) {
        // Get some event header fields
        EventBus.ChangeEventHeader header = event.ChangeEventHeader;
        // All the AgentWork records that were updated in this transaction. Could be
        multiple when updating due to User going offline with multiple work items assigned.
        List<String> recordIds = event.ChangeEventHeader.getRecordIds();
        System.debug('Received change event for ' +
header.entityName +

```

```

        ' for ' + header.recordIds + ' of type ' + header.changeType + ' operation.');
```

if (header.changetype == 'CREATE')

```

    {
        // New AgentWork record - only care about this if mixing External and
        // Omni-Channel routed work, as for external work
        // the external 3rd party application will have created the AgentWork record
        // so already knows about it.

    }
    // For update operations, we can get a list of changed fields
    else if (header.changetype == 'UPDATE') {
        // Updated record. Updates we care about are:
        // Status - main updates you care about are: 'Opened' - means it was accepted.
        // Any other state update is an end state and will indicate capacity has been released for
        // the user.
        // If mixing External and Internal work, you will get here for both. Need to
        // ensure it is an AgentWork Id that you care about
        // to determine that an externally routed work item is completed. If it is
        // internally routed, you will need to query the DB
        // to get the capacity that the work item consumed and therefore released (both
        // 'CapacityWeight' and 'CapacityPercentage').
        System.debug('List of all changed fields:');
        // Process work - remember to iterate all record Ids
        for (String field : header.changedFields) {
            if (null == event.get(field)) {
                System.debug('Deleted field value (set to null): ' + field);
            } else {
                System.debug('Changed field value: ' + field + '. New Value: '
                    + event.get(field));
            }
        }
    }
    else if (header.changetype == 'DELETE') {
        // Shouldn't ever get here for AgentWork
    }
}
}

```

Create AgentWork Records for External Routing

The external routing application creates AgentWork records to route work items to a rep in Omni-Channel.



Note: We strongly recommend not to use external routing and Omni-Channel routing simultaneously because they may assign the same work item to the same rep. This conflicting assignment could lead to complications while tracking consumed work capacity.

When the partner routing application receives new `PendingServiceRouting` creation events (where the routing type equals *External*), it uses the values in the event notification to determine what work needs to be routed to a rep. It makes a routing decision by creating AgentWork records. This API is existing functionality that partners use to query Salesforce.

The partner routing application uses these steps to generate and send the AgentWork records:

- Creates an AgentWork record using the IDs, such as the PendingServiceRouting ID and WorkItem ID, gathered from the PendingServiceRouting event notification. Keep track of the PSR ID and the WorkItem ID as they are required for creation of the AgentWork record.

- Stores the `AgentWork` records in the external routing application until they are ready to be routed to a rep.
- Executes Apex code to send the ready `AgentWork` record to Salesforce.

This Apex code sample creates an `AgentWork` record and assigns the work item to a specific rep.

```
AgentWork work = new AgentWork();
work.ServiceChannelId = '<ServiceChannelId>';
work.WorkItemId = '<WorkItemId>';
work.UserId = '<UserId>';
work.PendingServiceRoutingId = '<PendingServiceRoutingId>';
work.CapacityWeight = <Capacity Weight from PSR>;
insert work;
```

Transferring Work to a Specific Rep

When your routing requirements necessitate that work be directed to a specific rep rather than a queue, you can utilize the Preferred User functionality. This is particularly useful for maintaining continuity in customer service or handling high-priority escalations.

How it Works

To transfer work to a specific support rep, turn on Skills-Based and Direct-to-Agent Routing in Omni-Channel settings. Once these settings are enabled, when a Pending Service Routing (PSR) record is generated, the system populates a specific field to signal the routing engine:

- **Field:** PreferredUserId
 - **Logic:** The external routing engine should prioritize the ID found in this field.
 - **Outcome:** Instead of following standard load-balancing or queue-based logic, the engine attempts to assign the `AgentWork` record directly to the specified UserId.
-  **Note:** Ensure your external routing engine is configured to check for the presence of a PreferredUserId on the PSR record before executing its standard distribution algorithms. If this field is populated, it should override the default queue routing to ensure the work reaches the intended rep.

Expected Behavior for External Routing for Omni-Channel

Verify that the behavior you observe while testing and using your implementation of external routing matches the following expected behavior scenarios.

Agent accepts the work:

1. Chat visitor initiates a chat request from the external routing button.
2. `PendingServiceRouting` is created.
3. Partner is notified by a pushTopic event (EventType=Create, isPushed=false).
4. Partner creates `AgentWork` using the PSR.
5. Agent is routed the chat request (`AgentWork Status = Assigned`).
6. Agent accepts the chat request (`AgentWork Status = Accept`).
7. Omni-Channel deletes the `PendingServiceRouting` after Agent accepts the work.
8. Partner is notified by a pushTopic event (EventType=Delete).

Agent declines the work through Omni-Channel:

1. Agent declines the assigned `AgentWork`.
2. Salesforce updates the `PendingServiceRouting`.

3. Partner is notified by a pushTopic event (EventType=Update, LastDeclinedAgentSession=agent's session id in Chat (not the Salesforce session), isPushed=false).
4. Partner creates a new AgentWork using the updated PendingServiceRouting for rerouting.

Agent doesn't accept the work due to push time-out:

1. Existing PendingServiceRouting is updated.
2. Partner is notified by a pushTopic event (EventType=Update, PSR Fields updated: isPushed=false, LastDeclinedAgentSession=agent's liveagent session id).
3. Partner creates a new AgentWork for rerouting.

Agent transfers the work to an external routing queue:

1. New PendingServiceRouting for the transfer is created.
2. Partner is notified by a pushTopic event (EventType=Create, isTransfer=true, isPushed=false).
3. The routing process is repeated.

Agent transfers the work to another agent :

1. The PendingServiceRouting from the original chat request is deleted.
2. A new PendingServiceRouting isn't created when the work is transferred. Subscribe to AgentWork and LiveChatTranscript to determine whether the work was transferred to an agent.
3. Two AgentWorks are created for the LiveChatTranscript:
 - a. First AgentWork with the Status = *Opened*
 - b. Second AgentWork with the Status = *Assigned*
4. The LiveChatTranscript is updated with the Status = *In Progress* and the Owner = *second Agent*.
5. To determine if the Transfer to Agent has occurred, check that the second AgentWork isn't inserted into the same LiveChatTranscript as the first AgentWork.

 **Important:** We don't recommend using both external routing and Omni-Channel queue-based routing in the same implementation. If the same agent is in both queues, the agent's capacity could be exceeded. We don't have control over an agent's capacity in external routing. If you attempt this combination, there can be unknown issues.

Troubleshooting External Routing for Omni-Channel

If you encounter issues with your implementation of External Routing for Omni-Channel, try the following troubleshooting steps.

Recover from an External Routing Adaptor Restart

When the third-party adaptor recovers from restarting, it should leverage the durability feature of the Streaming API (since version 37.0) and replay from the last successful position of the PSR topic.

Reference the following code sample in Java.

```
// Register streaming extension
var replayExtension = new cometdReplayExtension();
replayExtension.setChannel(***<Streaming Channel to Subscribe to>***);
replayExtension.setReplay(<Event Replay Option>);
cometd.registerExtension('myReplayExtensionName', replayExtension);
```

For more information, see [Message Durability](#) in the *Streaming API Developer Guide*.

Recover from a Salesforce Data Recovery Instance

An org instance can be recovered from a Salesforce data center switch. The recovery process involves downtime, so all online agents must be logged out. All states maintained by the third-party adaptor, such as agent presence, aren't applicable and must be reset. The third-party adaptor should reinitialize as when it first subscribed to the topic.

Test the Client Solution

You can use the Streaming API to listen to CRUD events for `UserServicePresence` and `PendingServiceRouting`. For examples, see [Code Examples](#) in the *Streaming API Developer Guide*.

INDEX

O

Omni-Channel [1](#), [3–4](#), [8–10](#), [15–17](#)